Structs

Swift lets you design your own types in a few different ways, of which the most common are called structures, or just `structs`. `structs` can be given their own variables and constants, and their own functions, then created and used however you want.

If you have any programming experience you may be familiar with Object-Oriented Programming, which uses `classes`, we will be getting into `classes` after `structs`. Just know there are some common topics between the two.

Both `classes` and `structs` can:

1. Define properties to store values.
2. Define methods to provide functionality.
3. Define subscripts to provide access to their values using subscript syntax (the `[]` square brackets you use with collections).
4. Define `initializers` to set up their initial state.
5. Be extended to expand their functionality beyond their default implementation.
6. Conform to `protocols` to provide standard functionality of some kind.

Swift is a Protocol Oriented Programming language, also called POP. That means that it is centered mostly around `structs` and `protocols` as opposed to `classes` and class inheritance. This means that you should always start with a `struct` until you need functionality that is only supplied by `classes`. This includes if you are doing interoperability with Objective-C.

There are a couple of things I want to talk about in relation to why we should typically choose structs over classes, beyond the “It's what we were told to do”.

1. `structs` are value types.
2. `structs` are memory efficient.
3. `structs` are faster than `classes`.
4. `structs` have a built-in (Member-wise) `initializer`.

We are going to spend a lot of time talking about `structs` and `classes`, they are one of the most important topics in Swift. Programming can be really inefficient without them. You have been using them this entire time as well. Anytime you have used a `String`, `Int`, `Boolean`, `Double`, any of the collections, etc. You have been using a `struct`. All basic and complex types in swift are values types, implemented as `structs`.

A good way to think about what one of these is (`structs` and `classes`), would be to picture it as a blueprint. You create the basis of what you need, or a way to represent some data. Then you create an instance of that data, or actually represent it now.

It's essentially a way to visualize, manage, and manipulate data. All things considered, it would be a lot easier to show you than sit here and talk all day. Let's get started.

Syntax

Just like with everything else, there is some specific syntax with `structs`. `structs` you use the keyword `struct` to create them. Then you put the name; however, this time we are creating a new type, so we use `UpperCamelCase`. Then after the keyword and name, you use the `{}` curly braces to wrap everything you want included in your `struct`. Those things being `properties` and `methods`. We call these `properties` and `methods` the members of the `struct`.

```
struct House {
    var numberOfWindows: Int
    var hasFireplace: Bool
    let yearBuilt: Int
}
```

Notice that I am not assigning a value to any of my variables or constants, which in this context we call properties. I am also using Type Annotation. We are building a blueprint here, not a concrete implementation. So we don't need to put values, I would say most of the time you don't. We do have the option to in the right situation, though.

```
let myHome = House(numberOfWindows: 5, hasFireplace: false, yearBuilt:
2023)
```

When we go to use our blueprint, or create a house, we use some syntax you have seen before. The type followed by parents, and then you fill out the properties required by the type. Similar to parameters of a function.

Now that I have created a blueprint, I can use it to represent all kinds of different values. Like maybe, I own 3 houses, I can use the same blueprint to represent all of them as data.

```
let myVacationHome = House(
    numberOfWindows: 25,
    hasFireplace: true,
    yearBuilt: 1968,
)

let myApartment = House(
    numberOfWindows: 10,
    hasFireplace: false,
```

```
    yearBuilt: 2008,
)
```

Let's create another `struct`, and show off more pieces of it.

```
struct Car {
    let vin: String
    let make: String
    let model: String

    var isRunning = false

    func honk(times: Int = 1) {
        for _ in 0..<times {
            print("Beep!")
        }
    }

    mutating func start() {
        if !isRunning {
            print("You started the car")
            isRunning = true
        } else {
            print("The car is already running!")
        }
    }
}
```

Types can declare both `variables` and `constants` which we refer to as their properties. We declare these the same way we have declared all other ones, just now we put it inside these curly braces. Anytime you have a value that won't change after we create an instance, you should ALWAYS use a `constant`. Safe practice is to always start with a `constant` and only change it to a `variable` if you have to.

When we declare these `variables` and `constants` we can also assign default values, if we assume that a value will most of the time be the case.

Types can also declare functions, which are called methods in this context. We can reference the properties on our type within these functions by using the keyword `self` followed by a `.`. Or you can usually just skip the `self` and call the property directly. I use them interchangeably based on context; however, there will be times when you need to explicitly specify you are using `self`.

You will notice that on the `start` method of our `Car` struct we added a new keyword: `mutating`. `mutating` is a keyword used specifically on methods defined inside of `structs` and what it does is it tells swift that the method can modify the properties of an instance of the struct. In the `Car` example we set `isRunning = true` within the body of the `start` function, so we need to mark that function as `mutating`. In the `honk` method however, we do not change any properties, so it is not marked as `mutating`.

```
var myCar = Car(vin: "123ABC456", make: "Hyundai", model: "Venue")
myCar.honk()
print(myCar.isRunning)
myCar.start()
print(myCar.isRunning)
```

It's important to note that we can only use mutating functions on instances that are stored within a `variable` as it needs to change the value stored there.

Structs Are Value Types

You have probably heard me say that `structs` are value types at least a couple of times in this lecture so far. This is a **REALLY** important thing to remember because it comes down to the core functionality of swift. When something is a value type in Swift that means that when we pass the value around we make a copy of it. In other words value types are "copy by value". So if we put the value into another variable or constant, we are simply copying the value. Let's do a couple of examples to show how this works.

```
struct Thermostat {
    var temperature = 0
}

var house = Thermostat(temperature: 72)
var oven = house
oven.temperature = 400

/// Structs are value types, so what is going to be printed here?
print("House: \(house.temperature)")
print("Oven: \(oven.temperature)")
```

The output will be `house: 72 and oven: 400`. When we said that our oven variable was equal to our house variable, we are simply copying the value that was stored in house and then now we have two different thermostats.

Let's do another example.

```
struct Person {
    var name: String
    var age: Int

    mutating func birthdayParty() {
        age += 1
    }
}
```

So here we have a person `struct`, or a blueprint that can represent a person. Let's go ahead and create our buddy Joe.

```
var personA = Person(name: "Joe", age: 24)
var personB = personA

print(personA.name, personA.age)
print(personB.name, personB.age)
```

So far we have to variables with the same information. Let's go ahead and say that `personA` doesn't like the other `personB` and doesn't want to share the same name anymore. Then coincidentally, `personB` has a birthday.

```
personA.name = "Bobby"
personB.birthdayParty()

print(personA.name, personA.age) // Bobby, 24
print(personB.name, personB.age) // Joe, 25
```

So we made a copy of our `personA` and stored it in `personB`, but then we modified each of them, which had no effect on the other one, so we end up with `Bobby, 24` and `Joe, 25`.

Section One Of Assignment

Computed Properties

`structs` have what are called computed properties. A computed property is similar to a function that returns a value. Swift lets us create these dynamic values in a way that makes them feel like any other value.

> **Note**

If you have experience in other programming languages you may be familiar with writing `get<Property>()` methods. These follow that same idea.

```
struct Movie {
    let name: String
    let releaseYear: Int
    var reviewScore: Double

    var overallReview: String {
        switch reviewScore {
            case 0..<2:
                return "\(name) is a mediocre movie, released in \
(releaseYear)"
            case 2..<4:
                return "\(name) is an okay movie, released in \
(releaseYear)"
            case 4...5:
                return "\(name) is a top-tier movie, released in \
(releaseYear)"
            default:
                return "\(name) has no reviews, released in \
(releaseYear)"
        }
    }
}

let movie = Movie(name: "Shrek", releaseYear: 2001, reviewScore: 5)
print(movie.overallReview) // Shrek is a top-tier movie, released in
2001
```

Property Observers

Similar, in a way, to computed properties are things called property observers. These give us the ability to react to changes on a property. We are given two different options with property observers. We have `willSet` this one gets triggered after a change has been requested, but before the change has happened. There is also `didSet` which get triggered after the change has happened.

Let's say that I am creating a piece of software that is used to track progress of some kind of task, and it needs to update the user every time progress has been made. We could use property observers to trigger those notifications.

```
struct Progress {
    var task: String
    var amount: Int {
        willSet {
            print("\(task) is \(amount)% complete. Soon to be \
(newValue)%")
        }
        didSet {
            print("\(task) is \(amount)% complete. Updated from \
(oldValue)%")
        }
    }
}
```

When we are inside a `willSet` we can get access to what the value is going to be changed to via the `newValue` constant that `willSet` provides us. Since the `willSet` gets called before the value actually gets changed, you can access that value by just using its name!

When we are inside a `didSet` we can get access to what the value was before it got changed via the `oldValue` constant that `didSet` provides us. Since the `didSet` gets called after the value gets changed, you can access what it got changed to by just using the properties name!

```
var progress = Progress(task: "Loading Something...", amount: 0)
progress.amount = 20
progress.amount = 40
progress.amount = 80
progress.amount = 100
```

If you run the code from above, you will get a nice description of what is happening thanks to our `willSet` and `didSet`. Notice, though, that we don't get anything out when we first create our `Progress`. So these property observers only trigger when a value is changed, this doesn't include the first time that the value is set.

Now, let's do another example. Let's say that I am building a step counter, to keep track of my steps. We could use a `didSet` to let us know that we are progressing!

```
struct StepCounter {
    var totalSteps: Int = 0 {
        // You can use either willSet or didSet or even both!
        didSet {
            // Make sure that we added steps instead of removing.
```

```
            if totalSteps > oldValue {
                print("Added \(totalSteps - oldValue) steps!")
            }
        }
    }
}

var stepCounter = StepCounter()
stepCounter.totalSteps = 200
stepCounter.totalSteps += 100
```

Nonmutating Methods

We have talked a lot about `mutating` methods, but a lot of our `functions` within structs aren't going to be `mutating`. We only need to mark a method as `mutating` if it is actually going to modify the contents of our `struct`. Let's do an example of a `struct` that uses no `mutating` methods.

```
struct City {
    var population: Int
    var taxRate: Int

    func collectTaxes() -> Int {
        return population * taxRate
    }
}
let chicago = City(population: 2_659_000, taxRate: 8)
print(chicago.collectTaxes())
```

Since our `collectTaxes` function doesn't modify the contents of either of our properties, it doesn't need to be `mutating`. A good rule of thumb to follow here is to not mark any `methods` as `mutating`, until you need them to be.

`Initializers`

`structs` and `classes` both have what is called an `initializer` or an `init`. An `init` is what we use to actually create an `instance` of our type. It is just a really fancy `function` that we call to create our value, that is why the syntax is so similar between calling a function and creating an `instance` of something.

`structs` by default, have what is called a member-wise `initializer`. This is an `initializer` that is given to us by swift, that simply represents all the properties we have on our `struct`.

We can create our own `inits` but when we do it means we lose access to the one given to us by Swift.

```
struct User {
    var username: String
    var age: Int

    init() {
        self.username = ""
        self.age = 0
    }

    init(name: String) {
        self.username = name
        self.age = 0
    }

    init(named name: String, yearsOld age: Int) {
        self.username = name
        self.age = age
    }
}

let userOne = User()
let userTwo = User(name: "TheLegend27")
let userThree = User(named: "MalornManor", yearsOld: 24)
```

We can declare as many `inits` as we want, they just all need to have unique `signatures` similar to `functions`. `inits` follow the same rules as function signatures, but they don't have a return value as it will always be the type they are declared for, and they don't have names.

Lazy Variables

There are times when you are creating an object that has a property that may take a long time to make. You may not need to access that property for a while. Swift offers what is called a `lazy` property. Lazy properties are properties that don't actually get initialized until the first time they are accessed.

```
struct FamilyTree {
    init() {
        // Some really long operation
        print("Creating Family Tree")
```

```
    }
}

struct PersonInFamily {
    var name: String
    lazy var familyTree = FamilyTree()

    init(name: String) {
        self.name = name
    }
}

var bill = PersonInFamily(name: "Bill")
print("What is bills family tree?")
bill.familyTree // Creating Family Tree
```

As you can see even though that the “Creating Family Tree” print statement is called in the `FamilyTree` `init`, it doesn't get called when we create a `PersonInFamily`. The print statement only gets called when we try to access that `familyTree` property.

Static Properties and Methods

`Static` `properties` and `methods` are different from our regular ones. These properties and methods exist on the type itself, not on any instances of the type. Due to the fact that it exists separately from an instance of your object, it can only access other static properties and methods.

Let's say we have a `struct` named `Student`. We want to track how many students we have enrolled at our college. We could use a `static` `variable` to keep track of that.

```
struct Student {
    // We declare a static variable by prefacing the var keyword with
`static`
    static var classSize = 0

    var name: String
    var id: Int

    init(name: String, id: Int) {
        self.name = name
        self.id = id

        // We dont use self here because we are referencing the Type.
        Student.classSize += 1
```

```
    }

    static func getClassSize() -> Int {
        return classSize
    }
}
```

We can declare `static` `properties` and `methods` by prefacing the normal keywords with the `static` keyword. This will make them a member of the type, not a member of an `instance`. This means that when we try to use them, we need to use the Type instead of an instance. So we normally would write `Student.classSize` or `Student.staticGetClassSize()`.

```
let fred = Student(name: "Fred", id: 123)
print("\(fred.name) is a student with the id: \(fred.id)")
print(Student.classSize) // 1

let daphne = Student(name: "Daphne", id: 124)
// print(daphne.classSize)
print("\(daphne.name) is a student with the id: \(daphne.id)")
print(Student.classSize) // 2
```

If we tried to access `classSize` via `daphne` or `fred` directly, we couldn't as they don't have those properties. The `Student` type is what stores that information.

```
let shaggy = Student(name: "Shaggy", id: 125)
print("\(shaggy.name) is a student with the id: \(shaggy.id)")
print(Student.getClassSize())

let velma = Student( name: "Velma", id: 126 )
print("\(velma.name) is a student with the id: \(velma.id)")
print(Student.getClassSize())
```

Access Control

Last thing we need to cover is a topic called access control. Access control is a set of keywords that control how properties and methods can be accessed in our code. There are six different levels of Access Control. One of which is class specific, so we will talk about that in the classes lecture. The five that can be used across both are as follows:

1. `public` - This is saying that your code can be accessed from anywhere. This one primarily gets used for creating packages of code that are meant for people to use in their projects.

2. `internal` — This is the default access control, all your properties use this one unless specified otherwise. This says that anything within the same project can access this property or method.
3. `fileprivate` — This one says that a property or method is only accessible to code within the same file.
4. `private` — This one says that this property or method is only accessible within the object itself, and only by the code that the object was originally created with.
5. `private(set)` — This one is property specific, we use it to say that our property can be read from anywhere, but can only be written to privately.

Similar to how I have said, you should treat your `properties` and `methods` as `constants` or `non-mutating`. With access control we typically want to assume that a `property` or `method` will be `private`, until proven otherwise.

I want to formally cover that when we create our own types, they get the same treatment as other types. Meaning that I can have properties inside a `struct` that are my custom types, or have them be parameters to `functions`, or anything else we can do with our other types.

Let's create an example that will bring all of what we have covered together. We are wanting to make a program that keeps track of students in an application process. We want to track how far through the process they are, what impression they made in an interview, and then the general information about the student.

We are going to start by making an `enum` that helps us track the impression that the potential student left on their interviewer.

```
enum InterviewImpression {
    case reallyGood
    case good
    case okay
    case bad
    case reallyBad
}
```

Now we need to create a type that will represent a student's application in our system. We are going to do this in a couple of different parts.

```
struct StudentApplication {

}
```

The first thing that I want to add to my `StudentApplication` is a way to keep track of all the applications that we have put into our system. We can use a `static variable` to

keep track of that. I then want to make sure that we can limit access to that information, so we are going to make that `private`.

```
// Inside our existing struct...
struct SudentApplication {
    private static var numberOfStudentApplications = 0
}
```

Now that we have a way to keep track of all our applications, we need to make sure that we have basic student information on had. So that we can figure out who each application belongs to. Since we already have a `struct` that can represent that information, we can just use that. I want this information to be easily accessible by the rest of our program, but it shouldn't be changed after we submit the application. So I am going to leave it as an `internal` property, but make it a `constant`.

```
// Inside our exisitng struct...
struct StudentApplication {
    // ... Other Code
    let studentInfo: Student
}
```

The next thing I want information about for our application is their history at different schools. Primarily what schools they have gone to, and what their GPA was. We could represent this really well by using a `Dictionary` where the key would be the name of the school, and the value would be their GPA. This information may change if they complete high school after their application, for example; however, I don't want this information to be changed easily. So we are going to make it a `private(set)` so everyone can see what their history is. While only people with a certain access level can update that information.

```
// Inside our existing struct...
struct StudentApplication {
    // ... Other Code
    private(set) var studentSchoolHistory: [String: Double]
}
```

Now all of this information will be put into the system by the interviewer who interviews the student. We want to make sure that the interviewer puts in their thoughts on the interview. This information shouldn't need to be written to after the initial submission; however, it should be easily accessible. So we are going to leave it as `internal`, but make it a `constant`. We can then use that `enum` we created to represent how the potential student did.

```
// Inside our existing struct...
struct StudentApplication {
    // ... Other Code
    let interviewImpression: InterviewImpression
}
```

Now we need a way to keep track of how far along we are in the process. Luckily earlier we created that `Progress` struct to help us represent that information, so we can use that! This information is too raw to display to anyone, so we are going to make it `private` and then have a `function` that will give a more suitable version as output.

```
// Inside our existing struct...
struct StudentApplication {
    // ... Other Code
    private var applicationProgress: Progress
}
```

I think that covers all the information we require for our `StudentApplication` now we have to figure out what it can do, or what we can make it do. I am going to go in a similar order of how we created our information, we are going to create the methods that would use that information.

First thing we need is a way to see how many applications we have. We made the `numberOfStudentApplications` `static` `property` `private`, so we can now add a `function` that will check if we are allowed to access that information, and then give it back if we are. At this stage, we haven't really covered any ways to verify how we could make sure we are allowed to access that information, so we are going to use our imagination to support that.

```
// Inside our existing struct...
struct StudentApplication {
    // ... Other Code
    static func getNumberOfApplicants() -> Int {
        // Validate we are allowed to have this information
        // Then:
        return numberOfStudentApplications
    }
}
```

Notice how above we didn't need to specify `StudentApplication.numberOfStudentApplications`, like how we normally would have

to. This is because since both the `numberOfStudentApplications` and `getNumberOfApplicants` members are `static`, they can access each other directly.

The next piece of functionality I want to have our `StudentApplication` be able to do, is print out information about the application. This would include some of their basic information, how they did, and how far along in the process they are. Let's create that function.

```
// Inside our existing struct...
struct StudentApplication {
    func printStudentApplicationInformation() {
        // The first thing we need to do is get all our information
        // in a readable format. Let's start with our students
information:
        let studentInformation = "\(self.studentInfo.name) has the id:
\(studentInfo.id)"

        // The next piece of information I want to get is a string that
could
        // represent how they did on the interview.
        let impressionString = switch self.interviewImpression {
            case .reallyGood: "left a really good impression"
            case .good: "left a good impression"
            case .okay: "left an okay impression"
            case .bad: "left a bad impression"
            case .reallyBad: "left a really bad impression"
        }

        // Now we need a good way to represent their progress.
        let progressInfo = "\(self.applicationProgress.amount)%
complete with the application process"

        // Now we can tie it all together
        print("\(studentInformation) and \(impressionString). They are
\(progressInfo)")
    }
}
```

The above function works, but to be honest, its kind of messy. I feel like this one function is doing a lot of different things, and could be broken down into some smaller pieces. For example, each of the pieces of information could be its own function.

The above is a very common example of how private functions can be used. They allow us to break down our other functions into smaller pieces, while not exposing unnecessary

functionality to the outside world. Let's break that function down into its pieces and compare the code.

```
// Inside our existing struct...
struct StudentApplication {
    // Replace the previous printStudentApplicationInformation.
    func printStudentApplicationInformation() {
        print( "\(stringForStudentInformation()) and \
(stringForStudentImpression()). They are \
(stringForStudentApplicationProgress())" )
    }

    private func stringForStudentImpression() -> String {
        switch interviewImpression {
            case .reallyGood : return "left a really good impression"
            case .good : return "left a good impression"
            case .okay : return "left an okay impression"
            case .bad : return "left a bad impression"
            case .reallyBad : return "left a really bad impression"
        }
    }

    private func stringForStudentInformation() -> String {
        return "\(studentInfo.name) has the id: \(studentInfo.id)"
    }

    private func stringForStudentApplicationProgress() -> String {
        return "\(applicationProgress.amount)% complete with the
application process."
    }
}
```

Above we may not have less code overall, in fact we have a little more. However, this breaks down the `printStudentApplicationInformation` function into smaller easier to read pieces. So overall our code is benefiting from this approach.

The last thing I want to do with our `StudentApplication` is to add some inits to make it easier to use.

```
// Inside our existing struct...
struct StudentApplication {
    init(
        studentInfo: Student,
```

```
        schoolHistory: [String: Double],
        impression: InterviewImpression,
        progressAmount: Int = 0
    ) {
        self.studentInfo = studentInfo
        self.studentSchoolHistory = schoolHistory
        self.interviewImpression = impression
        self.applicationProgress = Progress(
            task: "College Application",
            amount: progressAmount
        )
    }

    // Lets make an init that will let us create our application with
    // the Student info directly instead of with an existing student
struct.
    init(
        studentName: String,
        studentID: Int,
        schoolHistory: [String: Double],
        impression: InterviewImpression,
        progressAmount: Int = 0
    ) {
        studentInfo = Student(name: studentName, id: studentID)
        studentSchoolHistory = schoolHistory
        interviewImpression = impression
        applicationProgress = Progress(
            task: "College Application",
            amount: progressAmount
        )
    }
}
```

Now that we have created out `StudentApplication`, let's try and make some `instances` of it, and try to use it!

```
let fredsApplication = StudentApplication(
    studentInfo: fred,
    schoolHistory: ["Crystal Cove High School": 3.567],
    impression: .reallyGood,
    progressAmount: 50
)
```

```
let velmasApplication = StudentApplication(
    studentInfo: velma,
    schoolHistory: ["Crystal Cove High School": 4.0],
    impression: .good,
    progressAmount: 90
)

let ronsApplication = StudentApplication(
    studentName: "Ron",
    studentID: 789,
    schoolHistory: ["Patrick Henry High School": 2.34],
    impression: .bad
)
```

It's super cool that we can make applications now, but we can't be hard coding in our applications every time they come through. Luckily, since we made `StudentApplication` a type, we can use it in an `Array`. Let's store the ones that we have made inside `Array`!

```
var studentApplications = [
    fredsApplication,
    velmasApplication,
    ronsApplication,
]

// Now we can go through all of them and get information about them!

for application in studentApplications {
    application.printStudentApplicationInformation()
}
```